
AI-Assisted Microloan Application Readiness Checker

Trade-off Report

Name Sun Hanyu

Course PE6201 Emerging AI Technologies

Programme MSc Enterprise Artificial Intelligence

Project type Individual End-of-Course Project

Final system: deterministic rules + project-defined RAG + Gemini 3.8 Flash + human review

Executive summary

I built a pre-check system for synthetic microloan applications. Its purpose is to help a Credit Operations Officer identify incomplete or internally inconsistent application materials before formal credit assessment. It does not score credit, approve or reject a loan, set an interest rate or limit, retrieve external credit information, or act on a customer.

My main design question was whether a hybrid system of deterministic Python rules plus a foundation model could detect missing information and semantic inconsistencies more effectively than a rule-only baseline. I kept exact and auditable checks in code and rented Gemini 3.8 Flash only for semantic interpretation of the free-text loan purpose. I owned the data generator, labels, orchestration, response validation, evaluation, failure handling, and governance boundary.

I evaluated three configurations on the same frozen 50-case synthetic test set. The rule-only baseline achieved precision 1.000 but recall 0.333, missing 20 of 30 flag-worthy cases. The earlier rules-plus-Gemini system achieved precision 0.857, recall 1.000, and F1 0.923. My selected rules-plus-project-RAG-plus-Gemini final candidate achieved precision, recall, and F1 of 1.000, detected all 15 student-authored handwritten semantic contradictions, and had a 10% manual-review rate. It passed my predefined recall target of 0.90, the instructor's suggested precision floor of 0.70, and the preregistered citation and fixed security-suite conditions.

After the frozen run, one external reviewer independently assessed the 15 handwritten cases before seeing system outputs, then reviewed the supporting citations. Blind output agreement was 15/15. Fourteen citations were fully supported, one was partially supported, and none was unsupported. The review strengthens the coursework evidence but does not turn one reviewer or the project-defined corpus into institutional bank validation.

The strongest result is not simply a higher score. The final RAG run removed the rule baseline's silent misses and returned valid retrieved-source identifiers for every response, but it increased prompt tokens and direct model cost, depends on a hosted provider, and is grounded only in project-defined guidance rather than real bank policy. Its perfect automated score on a reused synthetic set does not establish performance in a real lending population, and the separate independent review covered only the 15 handwritten cases. I therefore accept the RAG hybrid only as a reversible pre-check with visible evidence and human authority, not as an autonomous credit decision system.

1. Problem, user, and significance

My primary user is a Credit Operations Officer in a small bank or fintech company. The persona in my problem statement is Mei, who handles approximately 40 to 60 small-loan applications per day. Before sending an application to a senior credit officer, she checks whether required materials are present, whether structured values are internally consistent, and whether the stated loan purpose conflicts with the form.

I separate that problem-framing estimate from quantities this project has actually measured:

Pain indicator	Evidence status
Approximately 40-60 applications per day	Persona estimate from my submitted problem statement; not an observed lender statistic.
Minutes spent checking one application	Not measured.
Percentage of applications returned for correction or rework	Not measured.
Reviewer wage or actual labour cost	Not measured.
Time saved, rework avoided, or financial benefit after using the prototype	Not measured because I did not run a real workflow study.

The quantified pain is therefore limited to workload frequency in the persona. I use evaluation quality, model usage, latency, and scenario costs to analyse the prototype, but I do not present them as proof of a real bank's operational savings.

The problem I address is narrow: repetitive readiness checking can detect exact omissions, but semantic contradictions in free text can remain invisible. My project asks whether a hybrid system can identify these cases without expanding into creditworthiness or approval.

The system changes Mei's workflow by giving her one of four readiness outputs - **Complete**, **Missing Documents**, **Inconsistent Information**, or **Manual Review** - together with reasons and evidence fields. It is intended to focus attention on incomplete, contradictory, or ambiguous submissions. I do not claim a measured time saving because the project contains no observed operational study of a real lender.

The explicit non-use boundary is essential. The tool must not be used to infer default risk, assess affordability, recommend approval or rejection, change a price or limit, or make an external action. These decisions are excluded both because they are outside my research question and because a wrong output in lending can materially affect a person.

Closest existing tool and the remaining gap

The closest publicly documented commercial tool I found is [Google Cloud Document AI for Lending](#). Google presents it as tooling for mortgage and home-loan document processing and data capture. Its documented components include optical character recognition, form parsing, document classification and splitting, custom extraction, and identity-document processing. It therefore addresses an adjacent upstream task: converting uploaded lending documents into structured information.

My prototype starts after that extraction stage. It consumes already structured synthetic application fields and tests a narrower downstream question: whether deterministic checks plus bounded language-model interpretation can identify missing inputs and semantic contradictions between a free-text microloan purpose and the declared purpose category, then return one of four project-defined readiness outcomes with reasons and evidence. The official Google pages I reviewed describe document extraction, classification, splitting, and normalization, but do not document this exact microloan-purpose consistency and readiness contract. I treat this as a gap in the publicly documented workflow, not as proof that the commercial product could never be configured or extended to perform it. In a production architecture, a document-processing product could operate upstream while my checker applies the downstream readiness contract.

2. Why a hybrid system

I assigned each task to the simplest technique that fits it.

Deterministic code owns document-presence checks, required fields, invalid numeric values, exact income consistency, and contradictions between document flags and document-derived values. These checks are mechanically verifiable. An LLM would make them less predictable without adding useful capability.

Gemini owns a smaller semantic task: deciding whether the purpose text is too vague and whether its meaning contradicts the declared purpose category or another supplied field. This task requires language interpretation, and I did not have a real labelled corpus large enough to train a narrow classifier.

The first frozen comparison did not use retrieval-augmented generation. I subsequently selected a bounded RAG version as the final candidate and preregistered a separate evaluation before running it. It retrieves from the project's own readiness and human-review contracts, makes the source used for an explanation visible, and validates source identifiers. The corpus is still project-defined coursework guidance rather than real bank policy. I did not use an agent because one application requires one semantic judgement, not a variable multi-step loop with tools. Adding an agent would add latency, cost, irreversible-action risk, and more failure modes without buying capability for this problem.

I treated the graphical interface as a presentation layer rather than the core system. The proposal made Streamlit optional, while the required first version was one application, one rule check, one LLM check, and one structured result. I first completed the reproducible command-line system, evaluation harness, and guardrails, then added a thin Streamlit interface that calls the same tested functions without changing the evaluation path.

3. Build-versus-buy decision

I made the build-versus-buy decision layer by layer.

Layer	Decision	Reason
Interface and serving	Build with Streamlit	A thin local interface exposes the same tested Python paths and the scope boundary without duplicating decision logic.
Application schema and rules	Build	These encode the project-specific definition of readiness and must be exact and testable.
Orchestration and merger	Build	I need deterministic precedence, preservation of rule findings, and explicit fallback behaviour.
Model service	Rent Gemini 3.8 Flash through the <code>google-genai</code> SDK	Training or serving a foundation model is unnecessary for the bounded semantic task; the hosted service supplies that commodity capability.
RAG retrieval	Build	A bounded lexical retriever indexes only the project readiness and human-review contracts; it excludes labels and results and validates cited chunk identifiers.
Data and ground truth	Build	The project requires reproducible synthetic data, frozen labels, and a separate semantic challenge slice.
Evaluation and observability	Build	Precision, recall, slice metrics, latency, token use, retries, and failures are part of the system.
Governance boundary	Build	The prohibition on credit decisions and the human-review contract cannot be delegated to the provider.

I did not adopt a low-code workflow builder or hosted assistant builder, and I did not run a low-code comparison experiment. I went directly to code because the assessment depends on exact rule precedence, frozen input and label files, strict response schemas, bounded retry behaviour, leakage controls, unit tests, and reproducible batch evaluation. A visual builder could present a prompt quickly, but it would not remove the need to own and verify those project-specific controls. This is a design rationale, not evidence that every low-code alternative would fail.

Time-to-deploy is qualitative in this project. Python and Streamlit allowed me to reuse the tested core in a local demonstration interface, so the prototype-to-local-demo path is complete. I did not deploy it to a production server or measure engineering hours, and I did not implement authentication, institutional integration, service monitoring, or production data controls. I therefore do not claim a production deployment time.

The proposal initially named Gemini 2.5 Flash. During implementation, the live API reported that model unavailable to new users and recommended Gemini 3.8 Flash. I approved and recorded that version change on 27 September 2026 before completing the final evaluation. The final report therefore names the model actually used rather than silently retaining the proposal name.

4. Data and ground truth

I used synthetic data because real loan applications would introduce privacy, access, consent, and redistribution issues that this coursework project could not responsibly resolve. The generator and all final records are included in the repository. No real names, identity numbers, account numbers, or customer records are used.

The project produced 200 development cases, 50 validation cases, and a 50-case frozen final set. The final set contains:

Slice	Cases
Clean applications	20
Missing-document cases	5
Numeric or field contradictions	5
Ambiguous-purpose cases	5
Handwritten semantic contradictions	15

The instructor identified an evaluation-validity risk in my original plan: if one script planted every inconsistency, the rules might only detect patterns created by that script. I therefore used 15 scenarios that originated from my own Chinese examples, then translated and revised them in English with AI assistance. Their provenance is recorded. They contain contradictions that require semantic interpretation rather than the existing if/else rules, and I report them separately.

I froze applications and labels before the final model run and recorded their hashes. Labels are stored separately from application inputs. The model receives only declared income, requested amount, existing monthly liabilities, purpose category, and purpose text. It does not receive ground truth, evaluation-slice metadata, or document-presence flags.

I implemented leakage prevention before the frozen model run. Applications and labels are separate files, the semantic payload is constructed from an explicit five-field allowlist, and the tests check that label fields and slice metadata do not enter the prompt. Document-presence flags are evaluated by deterministic rules but are also excluded from Gemini's semantic prompt. I have no valid before-and-after leakage score because I never evaluated an intentionally leaky model version. Creating one now only to produce a comparison would be a retrospective experiment, not evidence from the frozen evaluation. I therefore report the implemented exclusion and its tests, not an invented performance change.

This design reduces leakage, but it does not remove the main limitation of synthetic data. The 15 cases satisfy the instructor's later request for student-written semantic contradictions. After the frozen RAG run, one external reviewer, who identified their role as a bank employee with finance education and prior financial-inclusion research, completed a two-stage review. The reviewer agreed with all 15 case outputs, rated 14 citations fully supported, and rated one partially supported. This is useful independent checking, but one reviewer is not institutional validation and the full set may still reflect my own assumptions about microloan operations. I therefore treat the result as evidence about this test contract, not about a real lender's population.

5. Implementation

The final RAG path is deliberately small:

```

application
  -> strict application validation
  -> deterministic rule checks
  -> prompt-injection guardrail
  -> bounded retrieval from project-defined contracts
  -> one structured Gemini semantic check with retrieved context
  -> strict response and citation validation
  -> deterministic merger
  -> readiness outputs, reasons, and evidence

```

When the guardrail detects a high-signal injection pattern, the system skips retrieval and Gemini and adds **Manual Review** while preserving deterministic findings. Otherwise, the retriever supplies only project-defined readiness and human-review text. The model must cite retrieved chunk identifiers, and code rejects identifiers that were not retrieved for that application.

The semantic response must contain exactly four fields: semantic issue codes, a manual-review boolean, a non-empty reason, and evidence fields. Code rejects unknown keys, unknown issue codes, duplicate codes,

unsupported evidence, and logical inconsistencies. For example, a no-issue response may not cite evidence fields, while an identified issue must cite them. An invalid response or exhausted API failure becomes **Manual Review**.

The merger never allows the LLM to erase a deterministic finding. Missing-document and deterministic inconsistency results remain present even if the semantic call fails. The LLM cannot create a credit decision because the output vocabulary contains only readiness statuses.

I implemented bounded retries only for 503/**UNAVAILABLE**. I disabled automatic SDK retries so that two retry layers could not multiply calls unexpectedly. Quota failures are logged separately and are not treated as model predictions. The run checkpoints after every finalized case, allowing it to resume without repeating completed applications.

The repository currently passes 33 unit tests. These cover data composition, leakage exclusions, application validation, rule behaviour, semantic schema, deterministic merger, failure fallback, token metadata, interface mapping, retrieval relevance, prompt grounding, citation validation, RAG evaluation metrics, and the fixed prompt-injection guardrail suite.

6. Evaluation design

I compared the rule-only baseline, the earlier non-RAG hybrid, and the final RAG hybrid on the same 50 frozen cases. A case is flag-worthy when its expected output is not only **Complete**. I calculated precision, recall, F1, true positives, false positives, true negatives, false negatives, and manual-review rate. For the final RAG candidate I also preregistered retrieval coverage, citation coverage, citation-identifier validity, prompt-injection suite performance, latency, token use, retries, failures, and direct model cost.

The release conditions were fixed before seeing the final hybrid result:

- recall at least 0.90;
- precision at least 0.70; and
- separate results for the 15 handwritten semantic contradictions.

Recall alone would be inadequate because a system could send every case to manual review and achieve perfect recall. Precision measures whether raised flags are concentrated on cases that actually require attention. Manual-review rate exposes whether the system achieves safety by abstaining too often.

I also measure abstention quality against the frozen labels. **Appropriate review capture** is the proportion of cases labelled as requiring manual review that the system actually routes to review. **Unnecessary review rate** is the proportion of clean cases sent to manual review. These measures distinguish useful abstention from indiscriminate escalation.

7. Results

System	Precision	Recall	F1	TP	FP	TN	FN	Manual review
Rule-only	1.000	0.333	0.500	10	0	20	20	0.000
Rules plus Gemini 3.8 Flash	0.857	1.000	0.923	30	5	15	0	0.220
Rules plus project RAG plus Gemini 3.8 Flash	1.000	1.000	1.000	30	0	20	0	0.100

The rule-only baseline correctly handled all ten deterministic problem cases but missed all five ambiguous-purpose cases and all 15 handwritten semantic contradictions. Its **Complete** outputs were silent failures because nothing in the output indicated that meaning had not been checked.

APP_0286 demonstrates this failure. The category was education, but the text said tuition and tutoring were already covered and the loan would first buy a used motorcycle. The frozen label was **Inconsistent Information**. The rule-only system returned **Complete**; the hybrid system detected the semantic contradiction.

The earlier non-RAG hybrid detected all 30 flag-worthy applications and all 15 handwritten semantic cases. Five clean cases were false positives. Four were conservative **Manual Review** fallbacks after 503 failures. The fifth, **APP_0278**, was a substantive model disagreement: the frozen label treated a debt-consolidation application with zero monthly liabilities as clean, while Gemini viewed the debt statement and zero liabilities as contradictory. I kept it as a false positive in that run because changing the label after seeing the prediction would invalidate the evaluation. The final RAG candidate classified **APP_0278** as clean, but the original non-RAG result remains unchanged in the historical comparison.

The abstention results distinguish the two model-assisted runs. In the earlier non-RAG run, overall manual review was 22%, appropriate-review capture was $5/5 = 100\%$, and four of 20 clean cases were unnecessarily reviewed after provider failures. In the final RAG run, all five expected review cases were again captured, no clean case was sent to review, and the overall manual-review rate was $5/50 = 10\%$. The reduction cannot be attributed to retrieval alone because the RAG run also had no provider failures.

The final RAG candidate passed the numerical release conditions. All 50 calls succeeded without retry, all 50 responses cited at least one retrieved chunk, and all 109 cited identifiers were valid for their cases. Appropriate-review capture was 100%, and no clean case was unnecessarily reviewed. In the subsequent independent review of the 15 handwritten cases, blind output agreement was 100%; 14 citations were fully supported, one was partially supported, and none was unsupported. The partial case exposed a taxonomy gap for room extensions. The result still does not justify production deployment because the data are synthetic and reused, only one reviewer participated, and the knowledge base is not real bank policy.

The fixed prompt-injection suite also met its preregistered condition: all six malicious cases were blocked before retrieval and the model call, while all six benign controls passed through. This establishes expected behaviour only for the fixed high-signal patterns; it does not demonstrate general resistance to novel or obfuscated attacks.

8. Cost-to-serve

The final RAG run produced 50 successful semantic responses without retry. Calls reported 47,693 input tokens and 83,006 total tokens. I therefore estimate 35,313 billed output-plus-thinking tokens.

Using the Gemini 3.8 Flash prices recorded for 27 September 2026 - USD 0.75 per million input tokens and USD 3.75 per million output or thinking tokens - the direct model cost attributable from returned usage metadata is USD 0.1682 for the 50-case RAG experiment, or USD 0.00336 per incoming application. This is a calculation from usage metadata, not a reconstruction of the provider invoice.

Direct model cost is not the full cost to serve. The observed RAG manual-review rate was 10%. Because the project has no real reviewer wage or review-time observation, I use the two explicit course scenarios rather than inventing bank data:

Scenario	Cost per incoming application before fixed cost
45-second light review at USD 40/hour	USD 0.053
8-minute heavier escalation at USD 45/hour	USD 0.603

These figures include direct model cost plus expected human fallback. They exclude monthly fixed cost, represented as F / V , because neither fixed monthly cost F nor monthly application volume V has been confirmed. The sensitivity analysis in the repository shows that human fallback dominates token cost. Therefore the economically important variable is not the API price alone; it is how often the system escalates and how expensive that review is.

9. Failures, controls, and responsible use

I applied the Class 6 diagnostic: can I tell from the output that a component failed, and can I undo its action?

I use Singapore's [Model Artificial Intelligence Governance Framework, Second Edition](#) as the directly relevant governance reference. I map its emphasis on human-centric, explainable and transparent AI to concrete project controls: a human checkpoint before formal credit assessment, visible reasons and evidence fields, frozen and

reproducible evaluation, logged model failures, and an explicit prohibition on automated credit decisions. I do not claim formal compliance or independent assurance; I use the framework to justify the design choices and identify what a real deployment would still need.

Semantic misclassification may be invisible, but every system output is reversible because it occurs before formal credit assessment and triggers no external action. I therefore monitor invisible failures through frozen evaluation and slice metrics, while preserving a human review window.

Failure	Implemented control	Residual limitation
Rules miss semantic contradictions	Hybrid semantic check and separately reported semantic slice	LLM can also be wrong.
Structurally or logically invalid LLM response	Strict code validation and Manual Review fallback	A valid structure does not prove factual correctness.
503 provider failure	At most three attempts, checkpoint, then Manual Review	Adds delay and review workload.
429 quota failure	Pause and log separately; never treat as prediction	Delays evaluation or operation.
Ground-truth leakage	Separate labels and minimized semantic payload	Synthetic design bias remains.
Runaway retry cost	One SDK attempt plus bounded project retry; log usage	A production monthly budget is not defined.
Automation bias	Show original fields, rule findings, reason, evidence, and API status	No real reviewer study has been run.
Prompt injection in purpose text	A deterministic high-signal guardrail runs before retrieval and Gemini; detected attacks skip the model and become Manual Review ; six fixed attacks and six benign controls pass	Obfuscated, paraphrased, multilingual, or unfamiliar attacks may evade pattern matching.

Human review has three defined parts. The window is after the readiness result and before formal credit assessment. The evidence includes relevant application fields, issue codes, reasons, cited evidence fields, deterministic findings, and semantic-call status. The Credit Operations Officer may confirm readiness, request correction or documents, keep the case in manual review, or override the readiness label with a reason. The officer may not use this tool to approve or reject a loan or change its terms.

10. The trade-off I accept

I accept API dependence, additional retrieval context, higher token cost, average model latency of about 3.88 seconds, and a 10% manual-review rate in exchange for eliminating the rule baseline's 20 false negatives and making the project source used by each model response auditable. This is defensible because the system is a pre-check: an invisible **Complete** result can move contradictory materials forward without warning, while an ambiguous case remains reversible through review.

I do not accept autonomous action. The system stops before credit assessment because its semantic failures are not reliably visible from a fluent output. Keeping the output reversible is the central governance choice, not a disclaimer added after implementation.

I also accept a rented-model dependency rather than training a model. At this scale, the measured direct API cost is small, while owning the data contract, evaluation, guardrails, and orchestration gives me control over the parts specific to the problem. The observed model-version change and provider failures show the cost of that dependency, so the model name, code hashes, retries, and results are all recorded.

11. Limitations and next version

The project has six material limitations.

First, all data are synthetic. The experiment does not establish prevalence, accuracy, time saving, or economic value in a real institution. Second, the independent check used one reviewer and is not formal institutional assurance. Third, the same 50 cases were reused to compare the final RAG candidate, so they are not a new unseen holdout.

Fourth, the citation review covered the 15 handwritten cases rather than all 50 cases and found one partially supported citation because the project taxonomy did not define room extensions. Fifth, prompt-injection coverage is limited to a fixed high-signal suite, and production drift monitoring remains incomplete. Sixth, the RAG corpus contains project-defined guidance rather than real lender policy.

A next version should use multiple reviewers, extend the taxonomy to cover construction and room extensions, test the system on appropriately governed representative data, broaden the adversarial prompt-injection suite, and monitor input distribution, output balance, override rate, reopened cases, delayed labels, and sliced accuracy. Those changes should be versioned and evaluated against a new frozen set rather than inserted into this completed experiment.

12. Conclusion

The experiment answers my narrow question. On the frozen synthetic dataset, the final rules-plus-project-RAG-plus-Gemini candidate detected missing information and semantic inconsistencies more effectively than rules alone. Recall increased from 0.333 to 1.000, precision was 1.000, and all 15 handwritten semantic contradictions were detected.

The result also shows why the LLM and retrieval layer must remain bounded. The RAG run added context and direct model cost, its perfect score is limited to a reused synthetic set, and its sources are not real policy. My final design is therefore a RAG hybrid, evidence-producing, reversible pre-check. It uses deterministic code for what can be verified, bounded project retrieval for explicit context, a rented model for language judgement, and a human for ambiguity and failure. That is the trade-off I would defend.

References

Supplied course material

- PE6201 Project Problem Statement, Sun Hanyu, 22 August 2026.
- PE6201 End-of-Course Project Milestone 1: Watch-outs.
- PE6201 Class 1 materials: technique selection and checkability.
- PE6201 Class 2 materials: stack ownership and evaluation.
- PE6201 Class 3 materials: structured prompting, evaluation, and token economics.
- PE6201 Class 4 materials: agent definition and agent failure modes.
- PE6201 Class 5 C2: cost-to-serve, break-even, and sensitivity.
- PE6201 Class 6 C1-C3: failure diagnosis, guardrails, human review, and final-project requirements.

External product sources

- Google Cloud, [Document AI for Lending](#), accessed 27 September 2026.
- Google Cloud, [Document AI overview](#), accessed 27 September 2026.
- Infocomm Media Development Authority, Singapore, [Model Artificial Intelligence Governance Framework, Second Edition](#), 2020.